

CSS Min() Max() Clamp() Functions

Complete Guide: From Basics to Advanced

CSS Guide

May 10, 2026

Contents

1	Introduction	2
1.1	What are Min(), Max(), and Clamp()?	2
1.2	Evolution of Responsive Design	2
1.2.1	Before These Functions	2
1.2.2	With Min(), Max(), Clamp()	3
1.3	Browser Support	3
2	Understanding Min() Function	4
2.1	What Does min() Do?	4
2.1.1	Basic Syntax	4
2.1.2	How min() Works	4
2.2	Common min() Use Cases	4
2.2.1	Use Case 1: Responsive Width	4
2.2.2	Use Case 2: Padding/Margin	5
2.2.3	Use Case 3: Font Sizes	5
2.3	Advanced min() Examples	5
2.3.1	Responsive Typography Scale	5
3	Understanding Max() Function	6
3.1	What Does max() Do?	6
3.1.1	Basic Syntax	6
3.1.2	How max() Works	6
3.2	Common max() Use Cases	6
3.2.1	Use Case 1: Minimum Font Size	6
3.2.2	Use Case 2: Minimum Spacing	7
3.2.3	Use Case 3: Minimum Container Size	7
3.3	Advanced max() Examples	7
3.3.1	Responsive Layout Component	7
4	Understanding Clamp() Function	8
4.1	What Does clamp() Do?	8
4.1.1	Basic Syntax	8
4.1.2	How clamp() Works	8
4.2	Common clamp() Use Cases	8
4.2.1	Use Case 1: Fluid Typography	8
4.2.2	Use Case 2: Responsive Spacing	9
4.2.3	Use Case 3: Responsive Dimensions	9

4.3	Advanced clamp() Examples	10
4.3.1	Complete Responsive Page Layout	10
5	Combining Functions	11
5.1	Min() with Max()	11
5.1.1	Creating Flexible Ranges	11
5.2	Multiple Function Combinations	11
5.2.1	Complex Responsive Values	11
5.3	With Calc()	12
5.3.1	Combining calc() and clamp()	12
6	Real-World Examples	13
6.1	Example 1: Responsive Article Layout	13
6.2	Example 2: Responsive Card Grid	14
6.3	Example 3: Responsive Navigation	14
7	Browser Support & Fallbacks	16
7.1	Current Support Status	16
7.2	Fallback Strategies	16
7.2.1	Progressive Enhancement with Media Queries	16
7.2.2	Combining with Media Queries	16
8	Best Practices	18
8.1	Key Recommendations	18
8.1.1	1. Use Clamp() for Most Responsive Sizing	18
8.1.2	2. Maintain Readable Min/Max Ratios	18
8.1.3	3. Use Viewport Units Carefully	18
8.1.4	4. Test Across All Breakpoints	19
8.1.5	5. Combine with CSS Grid minmax()	19
9	Common Patterns	20
9.1	Fluid Typography System	20
9.2	Responsive Container Pattern	21
10	Troubleshooting	22
10.1	Common Issues	22
10.1.1	Issue 1: Values Not Scaling Smoothly	22
10.1.2	Issue 2: Text Too Small on Mobile	22
10.1.3	Issue 3: Container Overflow	22
11	Summary & Quick Reference	24
11.1	When to Use Each Function	24
11.2	Quick Reference	24
11.2.1	Function Syntax	24
11.2.2	Common Viewport Unit Ranges	24
11.3	Complete Responsive Foundation	24

Chapter 1

Introduction

1.1 What are Min(), Max(), and Clamp()?

The `min()`, `max()`, and `clamp()` functions are powerful CSS tools that enable **responsive design without media queries**. These mathematical functions automatically adjust CSS property values based on flexible constraints, creating fluid and adaptive layouts.

```
/* Basic syntax */
width: min(100%, 500px);
font-size: max(16px, 2vw);
padding: clamp(1rem, 5%, 2rem);
```

These functions revolutionize responsive design by providing:

- Automatic sizing based on constraints
- Fewer media queries needed
- More maintainable code
- Better performance
- Fluid scaling

1.2 Evolution of Responsive Design

1.2.1 Before These Functions

```
/* Old approach: Required media queries for every size change */
.container {
  width: 90%;
  max-width: 1200px;
  font-size: 16px;
}

@media (max-width: 768px) {
  .container {
    width: 95%;
    font-size: 14px;
  }
}
```

```
    }  
}  
  
@media (max-width: 480px) {  
  .container {  
    width: 100%;  
    font-size: 12px;  
  }  
}
```

1.2.2 With Min(), Max(), Clamp()

```
/* Modern approach: Single line handles all sizes */  
.container {  
  width: min(90%, 1200px);  
  font-size: clamp(12px, 2.5vw, 24px);  
}
```

1.3 Browser Support

Modern browsers support all three functions:

- Chrome/Edge: v75+ (min/max), v72+ (clamp)
- Firefox: v75+ (min/max), v75+ (clamp)
- Safari: v11.1+ (min/max), v13.1+ (clamp)
- Opera: v62+
- All modern mobile browsers

Chapter 2

Understanding Min() Function

2.1 What Does min() Do?

The `min()` function returns the **smallest** value from a list of arguments. It's perfect for setting maximum sizes when you want something to take up available space but not exceed a limit.

2.1.1 Basic Syntax

```
/* Select the smaller of the two values */
property: min(value1, value2);

/* Can have multiple values */
property: min(value1, value2, value3);

/* Practical example */
width: min(100%, 500px);
```

2.1.2 How min() Works

Viewport Width	100%	500px	Result
200px	200px	500px	200px (smaller)
400px	400px	500px	400px (smaller)
600px	600px	500px	500px (smaller)
1200px	1200px	500px	500px (smaller)

2.2 Common min() Use Cases

2.2.1 Use Case 1: Responsive Width

```
/* Container never exceeds 1200px but uses full width on small screens */
.container {
  width: min(90vw, 1200px);
  margin: 0 auto;
}

/* Image doesn't exceed parent width */
```

```
img {  
  width: min(100%, 800px);  
  height: auto;  
}
```

2.2.2 Use Case 2: Padding/Margin

```
/* Padding reduces on small screens, never goes below 8px */  
.section {  
  padding: min(5vw, 40px);  
}  
  
/* Gap between items scales but has minimum */  
.grid {  
  gap: min(2rem, 4vw);  
}
```

2.2.3 Use Case 3: Font Sizes

```
/* Heading size scales but never exceeds 48px */  
h1 {  
  font-size: min(8vw, 48px);  
  line-height: 1.2;  
}  
  
/* Subtitle never gets too small */  
h2 {  
  font-size: min(5vw, 32px);  
}
```

2.3 Advanced min() Examples

2.3.1 Responsive Typography Scale

```
/* All text scales fluidly */  
body {  
  font-size: min(16px, 2.5vw);  
}  
  
h1 { font-size: min(48px, 8vw); }  
h2 { font-size: min(36px, 6vw); }  
h3 { font-size: min(28px, 4.5vw); }  
p { font-size: min(18px, 2.5vw); }
```

Chapter 3

Understanding Max() Function

3.1 What Does max() Do?

The `max()` function returns the **largest** value from a list of arguments. It's perfect for setting minimum sizes, ensuring content stays readable and accessible even on very small screens.

3.1.1 Basic Syntax

```
/* Select the larger of the two values */
property: max(value1, value2);

/* Can have multiple values */
property: max(value1, value2, value3);

/* Practical example */
font-size: max(16px, 2vw);
```

3.1.2 How max() Works

Viewport Width	16px	2vw	Result
200px	16px	4px	16px (larger)
400px	16px	8px	16px (larger)
800px	16px	16px	16px (equal)
1200px	16px	24px	24px (larger)

3.2 Common max() Use Cases

3.2.1 Use Case 1: Minimum Font Size

```
/* Text is always readable, minimum 16px */
body {
  font-size: max(16px, 1.5vw);
}

/* Headings scale up on large screens */
h1 {
```

```
    font-size: max(24px, 5vw);  
}
```

3.2.2 Use Case 2: Minimum Spacing

```
/* Margin always at least 16px, scales up */  
.box {  
    margin: max(1rem, 2vw);  
}  
  
/* Gap between items grows on bigger screens */  
.grid {  
    gap: max(20px, 3vw);  
}
```

3.2.3 Use Case 3: Minimum Container Size

```
/* Content container has minimum height */  
.hero {  
    min-height: max(400px, 100vh);  
}  
  
/* Element grows with screen but has minimum */  
.sidebar {  
    width: max(250px, 20vw);  
}
```

3.3 Advanced max() Examples

3.3.1 Responsive Layout Component

```
.card {  
    padding: max(20px, 3vw);  
    border-radius: max(8px, 1vw);  
    font-size: max(14px, 1.2vw);  
}  
  
.card h2 {  
    font-size: max(20px, 2.5vw);  
    margin-bottom: max(10px, 1.5vw);  
}  
  
.card p {  
    font-size: max(14px, 1.1vw);  
    line-height: max(1.4, 1.8);  
}
```

Chapter 4

Understanding Clamp() Function

4.1 What Does clamp() Do?

The `clamp()` function takes three arguments and returns a value within a **minimum and maximum range**. It's the most powerful function for truly fluid, responsive design that scales smoothly across all viewport sizes.

4.1.1 Basic Syntax

```
/* Three required arguments: minimum, preferred, maximum */  
property: clamp(MIN, PREFERRED, MAX);  
  
/* Practical example */  
font-size: clamp(16px, 2.5vw, 32px);
```

4.1.2 How clamp() Works

The value returned follows this logic:

- If `PREFERRED < MIN`: return `MIN`
- If `PREFERRED > MAX`: return `MAX`
- If `MIN ≤ PREFERRED ≤ MAX`: return `PREFERRED`

Viewport	Min	Preferred (2.5vw)	Max	Result
200px	16px	5px	32px	16px (below min)
400px	16px	10px	32px	10px (preferred)
800px	16px	20px	32px	20px (preferred)
1400px	16px	35px	32px	32px (exceeds max)

4.2 Common clamp() Use Cases

4.2.1 Use Case 1: Fluid Typography

```
/* Perfect for scalable headings */
h1 {
  font-size: clamp(28px, 5vw, 56px);
  line-height: clamp(1.2, 1.5, 1.8);
}

h2 {
  font-size: clamp(20px, 3.5vw, 40px);
}

body {
  font-size: clamp(14px, 2vw, 18px);
}
```

4.2.2 Use Case 2: Responsive Spacing

```
/* Padding scales smoothly */
.container {
  padding: clamp(20px, 5vw, 60px);
}

/* Gap between items */
.grid {
  gap: clamp(1rem, 3vw, 3rem);
}

/* Margin that adapts */
.section {
  margin: clamp(2rem, 8vw, 6rem);
}
```

4.2.3 Use Case 3: Responsive Dimensions

```
/* Width adapts but stays within bounds */
.container {
  width: clamp(300px, 90vw, 1200px);
}

/* Height with constraints */
.hero {
  height: clamp(300px, 100vh, 800px);
}

/* Border radius scales */
.box {
  border-radius: clamp(4px, 1vw, 16px);
}
```

4.3 Advanced clamp() Examples

4.3.1 Complete Responsive Page Layout

```
body {  
  font-size: clamp(14px, 2vw, 18px);  
  line-height: clamp(1.4, 2vh, 1.8);  
}  
  
h1 {  
  font-size: clamp(28px, 6vw, 64px);  
  margin-bottom: clamp(1rem, 4vw, 3rem);  
}  
  
h2 {  
  font-size: clamp(22px, 4vw, 44px);  
  margin-bottom: clamp(0.75rem, 3vw, 2rem);  
}  
  
h3 {  
  font-size: clamp(18px, 3vw, 32px);  
  margin-bottom: clamp(0.5rem, 2vw, 1.5rem);  
}  
  
.container {  
  width: clamp(280px, 90vw, 1200px);  
  padding: clamp(1rem, 4vw, 3rem);  
  margin: 0 auto;  
}  
  
.grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
  gap: clamp(1rem, 3vw, 2rem);  
}  
  
.card {  
  padding: clamp(1rem, 3vw, 2rem);  
  border-radius: clamp(4px, 1vw, 12px);  
}  
  
button {  
  padding: clamp(8px, 1vw, 16px) clamp(12px, 2vw, 24px);  
  font-size: clamp(14px, 1.5vw, 18px);  
  border-radius: clamp(4px, 0.5vw, 8px);  
}
```

Chapter 5

Combining Functions

5.1 Min() with Max()

5.1.1 Creating Flexible Ranges

```
/* Width is at least 300px, at most 80vw */
width: min(max(300px, 80vw), 100%);

/* More readable alternative: use clamp instead */
width: clamp(300px, 80vw, 100%);
```

5.2 Multiple Function Combinations

5.2.1 Complex Responsive Values

```
/* Padding that's responsive and bounded */
.section {
  padding: max(1rem, min(5vw, 3rem));
}

/* Font size with multiple constraints */
h1 {
  font-size: max(
    24px,
    min(8vw, 64px)
  );
}

/* Line height combination */
body {
  line-height: max(1.4, min(1.5vw, 1.8));
}
```

5.3 With Calc()

5.3.1 Combining calc() and clamp()

```
/* Mix calculation with clamping */
.container {
  width: clamp(300px, calc(100% - 2rem), 1200px);
  padding: clamp(1rem, calc(2vw + 0.5rem), 3rem);
}

/* Complex spacing calculation */
.section {
  margin: clamp(1rem, calc(5vw - 0.5rem), 4rem);
}
```

Chapter 6

Real-World Examples

6.1 Example 1: Responsive Article Layout

```
/* Article container */
article {
  max-width: clamp(280px, 90vw, 800px);
  margin: 0 auto;
  padding: clamp(1rem, 5vw, 2rem);
}

article h1 {
  font-size: clamp(28px, 6vw, 48px);
  line-height: clamp(1.2, 1.4, 1.6);
  margin-bottom: clamp(1rem, 4vw, 2rem);
}

article h2 {
  font-size: clamp(22px, 4vw, 36px);
  margin-top: clamp(2rem, 6vw, 3rem);
  margin-bottom: clamp(1rem, 3vw, 1.5rem);
}

article p {
  font-size: clamp(14px, 2vw, 18px);
  line-height: clamp(1.5, 2vh, 1.8);
  margin-bottom: clamp(1rem, 3vw, 1.5rem);
}

article img {
  width: 100%;
  max-width: clamp(200px, 90vw, 800px);
  height: auto;
  margin: clamp(1rem, 3vw, 2rem) 0;
}

article blockquote {
  padding: clamp(1rem, 3vw, 2rem);
  border-left: clamp(2px, 0.5vw, 4px) solid #667eea;
  font-size: clamp(16px, 1.8vw, 20px);
}
```

```
    font-style: italic;
}
```

6.2 Example 2: Responsive Card Grid

```
.grid-container {
  width: clamp(280px, 90vw, 1400px);
  margin: 0 auto;
  padding: clamp(1rem, 3vw, 2rem);
}

.grid {
  display: grid;
  grid-template-columns: repeat(
    auto-fit,
    minmax(clamp(250px, 20vw, 350px), 1fr)
  );
  gap: clamp(1rem, 2vw, 2rem);
}

.card {
  background: white;
  border-radius: clamp(8px, 1vw, 16px);
  padding: clamp(1rem, 2vw, 2rem);
  box-shadow: 0 clamp(2px, 0.5vw, 4px)
              clamp(8px, 1.5vw, 12px)
              rgba(0, 0, 0, 0.1);
}

.card h3 {
  font-size: clamp(18px, 2.5vw, 24px);
  margin-bottom: clamp(0.5rem, 1vw, 1rem);
}

.card p {
  font-size: clamp(14px, 1.5vw, 16px);
  line-height: clamp(1.4, 1.6, 1.8);
  color: #666;
}

.card button {
  margin-top: clamp(1rem, 2vw, 1.5rem);
  padding: clamp(8px, 1vw, 12px)
           clamp(16px, 2vw, 24px);
  font-size: clamp(14px, 1.3vw, 16px);
  border-radius: clamp(4px, 0.5vw, 8px);
}
```

6.3 Example 3: Responsive Navigation

```
nav {
  padding: clamp(0.75rem, 2vw, 1.5rem);
  background: #2c3e50;
}

nav .logo {
  font-size: clamp(18px, 2.5vw, 28px);
  font-weight: bold;
  color: white;
}

nav ul {
  display: flex;
  gap: clamp(1rem, 2vw, 2rem);
  list-style: none;
  padding: 0;
}

nav a {
  font-size: clamp(14px, 1.5vw, 18px);
  color: white;
  text-decoration: none;
  padding: clamp(0.5rem, 1vw, 1rem)
        clamp(1rem, 1.5vw, 1.5rem);
  border-radius: clamp(4px, 0.5vw, 8px);
}

nav a:hover {
  background: rgba(255, 255, 255, 0.1);
}

.hamburger {
  display: none;
  font-size: clamp(18px, 2vw, 24px);
}

@media (max-width: 768px) {
  .hamburger {
    display: block;
  }

  nav ul {
    display: none;
  }
}
```

Chapter 7

Browser Support & Fallbacks

7.1 Current Support Status

Browser	min()/max()	clamp()	Status
Chrome	v75+	v72+	Full Support
Firefox	v75+	v75+	Full Support
Safari	v11.1+	v13.1+	Full Support
Edge	v75+	v72+	Full Support
Opera	v62+	v60+	Full Support
IE 11	Not Supported	Not Supported	

7.2 Fallback Strategies

7.2.1 Progressive Enhancement with Media Queries

```
/* Default: old browsers without these functions */
.container {
  width: 90%;
  max-width: 1200px;
  padding: 20px;
  font-size: 16px;
}

/* Enhanced: modern browsers with min/max/clamp */
@supports (width: clamp(1px, 1%, 1px)) {
  .container {
    width: clamp(280px, 90vw, 1200px);
    padding: clamp(1rem, 3vw, 2rem);
    font-size: clamp(14px, 2vw, 18px);
  }
}
```

7.2.2 Combining with Media Queries

```
/* Works for older browsers + simpler layout */
.card {
```

```
padding: 1rem;
font-size: 16px;
}

/* Modern responsive approach */
.card {
padding: clamp(1rem, 2vw, 2rem);
font-size: clamp(14px, 1.5vw, 18px);
}

/* Fallback for IE 11 */
@media (max-width: 480px) {
    .card {
padding: 0.75rem;
font-size: 14px;
    }
}
```

Chapter 8

Best Practices

8.1 Key Recommendations

8.1.1 1. Use Clamp() for Most Responsive Sizing

Clamp is the most powerful and should be your default choice:

```
/* Good: Simple and powerful */  
font-size: clamp(16px, 2.5vw, 32px);  
  
/* Still valid but less flexible */  
font-size: max(16px, 2.5vw);
```

8.1.2 2. Maintain Readable Min/Max Ratios

```
/* Good: Clear hierarchy and readable */  
h1 { font-size: clamp(28px, 5vw, 56px); }  
h2 { font-size: clamp(22px, 3.5vw, 44px); }  
h3 { font-size: clamp(18px, 2.5vw, 32px); }  
  
/* Bad: Inconsistent and hard to maintain */  
h1 { font-size: clamp(10px, 10vw, 100px); }  
h2 { font-size: clamp(5px, 3vw, 20px); }
```

8.1.3 3. Use Viewport Units Carefully

```
/* Good: Balanced scaling */  
font-size: clamp(16px, 2vw, 32px);  
  
/* Bad: Too aggressive scaling */  
font-size: clamp(10px, 10vw, 40px); /* Text changes drastically */  
  
/* Bad: Overly conservative */  
font-size: clamp(14px, 0.5vw, 16px); /* Barely scales */
```

8.1.4 4. Test Across All Breakpoints

```
/* Test these viewport sizes: */
/* Mobile: 320px, 375px, 480px */
/* Tablet: 768px, 1024px */
/* Desktop: 1440px, 1920px, 2560px */

/* Verify that: */
/* 1. Text remains readable at all sizes */
/* 2. No awkward jumps in scaling */
/* 3. Content doesn't overflow at any size */
/* 4. Spacing feels proportional */
```

8.1.5 5. Combine with CSS Grid minmax()

```
/* Perfect combination for responsive grids */
.grid {
  display: grid;
  grid-template-columns: repeat(
    auto-fit,
    minmax(clamp(250px, 20vw, 350px), 1fr)
  );
  gap: clamp(1rem, 2vw, 2rem);
}
```

Chapter 9

Common Patterns

9.1 Fluid Typography System

```
/* Create a complete typographic scale */
:root {
  /* Base size scales from 14px to 18px */
  --text-sm: clamp(12px, 1.5vw, 14px);
  --text-base: clamp(14px, 2vw, 18px);
  --text-lg: clamp(16px, 2.5vw, 22px);
  --text-xl: clamp(18px, 3vw, 28px);

  /* Headings scale dramatically */
  --h3: clamp(18px, 2.5vw, 32px);
  --h2: clamp(22px, 3.5vw, 44px);
  --h1: clamp(28px, 5vw, 56px);

  /* Spacing scale */
  --space-xs: clamp(0.25rem, 0.5vw, 0.5rem);
  --space-sm: clamp(0.5rem, 1vw, 1rem);
  --space-md: clamp(1rem, 2vw, 1.5rem);
  --space-lg: clamp(1.5rem, 3vw, 2rem);
  --space-xl: clamp(2rem, 4vw, 3rem);
}

body {
  font-size: var(--text-base);
  line-height: clamp(1.4, 2vh, 1.8);
}

h1 { font-size: var(--h1); }
h2 { font-size: var(--h2); }
h3 { font-size: var(--h3); }

.section {
  padding: var(--space-lg);
  margin-bottom: var(--space-xl);
}
```

9.2 Responsive Container Pattern

```
/* One-size-fits-all container */
.container {
  width: clamp(280px, 90vw, 1200px);
  margin-left: auto;
  margin-right: auto;
  padding: clamp(1rem, 3vw, 2rem);
}

/* Works for all content */
.hero .container {
  height: clamp(300px, 100vh, 800px);
}

.section .container {
  padding: clamp(2rem, 5vw, 4rem);
}
```

Chapter 10

Troubleshooting

10.1 Common Issues

10.1.1 Issue 1: Values Not Scaling Smoothly

Problem: Noticed jumps or unnatural scaling.

Solution:

```
/* Calculate viewport unit percentage correctly */
/* 2vw is good for body text */
/* 3-5vw is good for headings */
/* 0.5vw is good for small elements */

/* Test the math: */
/* At 1440px (avg desktop): 2vw = 28.8px */
/* At 360px (avg mobile): 2vw = 7.2px */

/* If scaling seems wrong, recalculate vw percentage */
```

10.1.2 Issue 2: Text Too Small on Mobile

Problem: Text falls to minimum value on small screens.

Solution:

```
/* Increase minimum value or adjust vw */
/* Wrong: Minimum too low */
font-size: clamp(10px, 2vw, 32px);

/* Better: Higher minimum */
font-size: clamp(14px, 2vw, 32px);

/* Or: Increase vw contribution */
font-size: clamp(12px, 3vw, 32px);
```

10.1.3 Issue 3: Container Overflow

Problem: Content overflows on certain screen sizes.

Solution:

```
/* Always consider safe margins */
.container {
  width: clamp(280px, 90vw, 1200px);
  /* 90vw leaves 5% on each side */
}

/* With padding, content truly safe */
.container {
  width: clamp(280px, 100vw, 1200px);
  padding: clamp(1rem, 2vw, 2rem);
  /* Padding is subtracted from width */
}

/* Or use box-sizing */
.container {
  box-sizing: border-box;
  width: clamp(280px, 100vw, 1200px);
  padding: clamp(1rem, 2vw, 2rem);
}
```

Chapter 11

Summary & Quick Reference

11.1 When to Use Each Function

Function	Purpose	Best Use
min()	Get smaller value	Max widths, caps sizes
max()	Get larger value	Min sizes, ensures readability
clamp()	Range between min/max	Fluid responsive design

11.2 Quick Reference

11.2.1 Function Syntax

```
/* min(): Returns smallest value */
width: min(100%, 500px);

/* max(): Returns largest value */
font-size: max(16px, 2vw);

/* clamp(): Returns value between min and max */
padding: clamp(1rem, 3vw, 2rem);
```

11.2.2 Common Viewport Unit Ranges

Use Case	Recommended vw	Example
Body text	1.5-2vw	clamp(14px, 2vw, 18px)
Small headings (h3)	2.5-3vw	clamp(18px, 2.5vw, 32px)
Medium headings (h2)	3-4vw	clamp(22px, 3.5vw, 44px)
Large headings (h1)	4-6vw	clamp(28px, 5vw, 56px)
Spacing	2-4vw	clamp(1rem, 3vw, 2rem)

11.3 Complete Responsive Foundation

```
/* Copy this as your starting point */
:root {
```

```
--text-base: clamp(14px, 2vw, 18px);
--text-lg: clamp(16px, 2.5vw, 22px);
--h3: clamp(18px, 2.5vw, 32px);
--h2: clamp(22px, 3.5vw, 44px);
--h1: clamp(28px, 5vw, 56px);
--space: clamp(1rem, 3vw, 2rem);
}

body {
  font-size: var(--text-base);
  line-height: clamp(1.4, 2vh, 1.8);
}

.container {
  width: clamp(280px, 90vw, 1200px);
  margin: 0 auto;
  padding: var(--space);
}

h1 { font-size: var(--h1); }
h2 { font-size: var(--h2); }
h3 { font-size: var(--h3); }
```

Thank you for reading!

For more information, visit [MDN Web Docs](#) and [web.dev](#)